POLYDIM: A Transformer-Native Algebraic Programming Language for High-Dimensional Latent Computing

~~It's the geometry, stupid.

Foundations of Cognitive Programming: POLYDIM as a First Formalization

Abstract

Modern AI architectures operate natively in continuous high-dimensional geometric spaces ($R^N, N \ge 10,000$), yet current programming paradigms force them to communicate via discrete sequential text channels — a mismatch we call the *impedance gap*. This paper posits that addressing this gap requires more than an engineering framework; it requires a new theoretical foundation: **Cognitive Programming**. We propose a formal discipline for specifying, verifying, and composing computational processes whose behavior emerges from the evolution of internal representations directed by goals.

Within this broader theoretical framework, we present **POLYDIM** as a first formal case study. POLYDIM is a language whose fundamental program unit is a geometric transformation $T: R^N \to R^N$. We prove four algebraic theorems establishing POLYDIM's formal foundations: associativity of composition (T1), linearity of superposition (T2), functoriality of projection to executors (T3), and uniqueness of fixed points (T4). We formalize a two-layer architecture separating the invariant algebraic core (COMPOSE, MIX, FIXPOINT, PROJECT) from architecture-specific implementation primitives (ATTEND for transformers, RECUR for SSMs). By interpreting programming as the formal orchestration of latent spaces, POLYDIM fills the gap between empirical latent communication research and rigorous formal language design.

PART I: The Cognitive Programming Framework

1. The Object of Study

For most of its history, Computer Science studied how to describe computational processes through algorithms executed on discrete machines. This formulation produced one of the most successful scientific theories ever developed.

However, the emergence of systems capable of learning representations, using tools, planning, and coordinating complex processes introduces a new phenomenon. For the first time, a significant part of a system's useful behavior no longer depends exclusively on the explicit algorithm written by the programmer. It depends on the evolution of internal representations under constraints and goals.

This constitutes the starting point of **Cognitive Programming**.

- The classical theory answers: *How do we execute an algorithm correctly?*
- The new theory answers: *How do we formally specify a cognitive process whose trajectory is not fully determined prior to execution?*

2. The Tuple of a Cognitive Program

A program ceases to be a list of instructions and becomes a formal system: **$P = (S, G, T, O, C, \Pi)$**

Where:

1. **S (Cognitive State):** The complete representation relevant to deliberation.
2. **G (Goal):** That which defines an acceptable evolution.
3. **T (Transformation):** Operator that modifies the state ($T: S \to S$).
4. **O (Observer):** Function that interprets the state into a meaning space ($O: S \to M$).
5. **C (Constraint):** Condition that must be preserved ($C(S) = true$).
6. **Π (Policy):** Rule for selecting the next transformation.

3. The Seven Axioms

1. **Axiom I:** Every program possesses a representational state.
2. **Axiom II:** Every state can be observed.
3. **Axiom III:** Every transformation produces another valid state.
4. **Axiom IV:** Transformations are composable.
5. **Axiom V:** Execution pursues explicit goals.
6. **Axiom VI:** Every execution can be evaluated against constraints.
7. **Axiom VII:** The policy Π can be modified during execution (deliberation).

4. The Cognitive Preservation Theorem

The greatest mathematical challenge of the discipline consists of proving under what conditions a reasoning process preserves the original goal G without violating constraints C. Solving this theorem is equivalent to mathematically eradicating "hallucination" and "goal drift".

PART II: The POLYDIM Formalization

5. Formal Calculus and Algebraic Primitives

POLYDIM acts as the experimental vehicle for the Cognitive Programming framework, specifically mathematizing the Transformation (T) and Observer (O) layers of the tuple.

5.1 State, Position, and Transformation

Definition (POLYDIM State). A state is a triple: $S = (V, D, A)$ where $V \in R^N$ (state

vector), D is a finite set of native observer subspaces, and A is the activation function.

Definition (Position). A position P is a pair: $P = (g, S)$ where $g \in R^N$ is the GEO_ID (invariant under all admissible transformations).

Definition (Transformation). A transformation T is a function $T: S \to S$. Admissible transformations modify V and may update A as a function of the new V, while leaving D fixed.

5.2 The Two-Layer Architecture

Algebraic Layer (permanent, architecture-independent):

- `COMPOSE(T1, T2)` = $T_2 \circ T_1$
- `MIX(a, T1, b, T2)` = $a \cdot T_1 + b \cdot T_2$
- `FIXPOINT(T, e)` = iterate until convergence
- `PROJECT(T, E)` = $\pi_E(T(s))$

Implementation Layer (architecture-specific):

- `ATTEND(Q, K, V, s)` = $softmax(QK^T/\sqrt{d})V$ (Transformer family)
- `RECUR(A, B, C, h, x)` = SSM family

6. Four Foundational Theorems

Theorem 1 (Associativity of COMPOSE): Standard property of function composition.

Theorem 2 (Linearity of MIX): If $T_1, T_2 \in Lin(R^N)$, then $MIX(a, T_1, b, T_2) \in Lin(R^N)$.

Theorem 3 (PROJECT is a Functor): For each executor E, $PROJECT_E: G \to E_E$ satisfies the two functor axioms. Proved unconditionally for three contracts: COMPILE, EXPORT, and RENDER (via the Flutter Algebraic Isomorphism).

Theorem 4 (Uniqueness of Fixed Point — Banach): If T is a contraction, there exists a unique fixed point $s^$ *such that* $T(s^) = s^*$.

7. Identity and Interoperability (The ALIGN Protocol)

Reading a message is mathematically indistinguishable from computing an attention layer. This eliminates the impedance mismatch that plagues JSON-RPC. Given two latent spaces IA_A and IA_B, the ALIGN protocol uses Orthogonal Procrustes to map transformations directly between models without text serialization.

Conclusion

If over the coming decades the object of study ceases to be solely the algorithm and expands to include verifiable cognitive processes, the discipline will evolve. POLYDIM is not the final theory, but the first formal case study demonstrating that the representational layer can be mathematically

rigorous. Empirical work on latent communication confirms that bypassing text serialization is feasible; POLYDIM provides the formal specification layer that makes this approach composable, portable, and analyzable.

(Note: Extended engineering details, binary formats, VM implementation, and SOTA comparisons are available in the supplementary material and Github Repository).